BLOG

Tryhackme

Abstract

Identify vulnerabilities, gaining access and capture the Penetration test of Blog lab identified critical vulnerabilities, gained access, escalated privileges, and captured the flag through exploitation

Haider Sultan

@gmail.com

Table of Contents

1. Confidentiality Statement 3
2. Disclaimer 3
3. Contact Information 3
4. Assessment Overview 3
5. Assessment Components 4
   5.1. Internal Penetration Test 4
6. Finding Severity Ratings 5
7. Risk Factors 5
   7.1. Likelihood 5
   7.2. Impact 5
8. Scope 5
   8.1. Scope Exclusions 6
   8.2. Client Allowances 6
9. Executive Summary 6
   9.1. Scoping and Time Limitations 6
   9.2. Testing Summary 6
   9.3. Tester Notes and Recommendations 7
   9.4. Key Strengths and Weaknesses 8
10. Vulnerability Summary & Report Card 9
   10.1. Internal Penetration Test Findings 9
11. Technical Findings 10
   Internal Penetration Test Findings 10
   11.1. Finding IPT: Privilege Escalation via Writeable script in Cron Job 10
   11.2. Finding IPT: Hardcoded Database Credentials 10
   11.3. Finding IPT: Insecure Permissions on Web Files 10

1. Confidentiality Statement

This document written by **Haider Sultan** contains confidential and privileged information from a security assessment conducted for **TryHackMe**. The information contained within this report is strictly confidential and intended solely for authorized personnel; it may not be distributed, copied, or disclosed to any third party without explicit written consent from **TryHackMe** and **Haider Sultan**.

2. Disclaimer

A penetration test represents a snapshot in time, with the findings and recommendations based solely on the information collected during the assessment period. Any changes or modifications made after this period are not reflected in the results.

Haider Sultan prioritized the assessment to focus on identifying the most vulnerable security controls that an attacker would likely exploit.

It is recommended that similar assessments be conducted annually, either by internal teams or independent third-party assessors, to ensure the ongoing effectiveness and improvement of security controls.

3. Contact Information

Name	Title	Contact Information
Haider Sultan	**Penetration Tester**	**Phone:** **Email: @gmail.com**

4. Assessment Overview

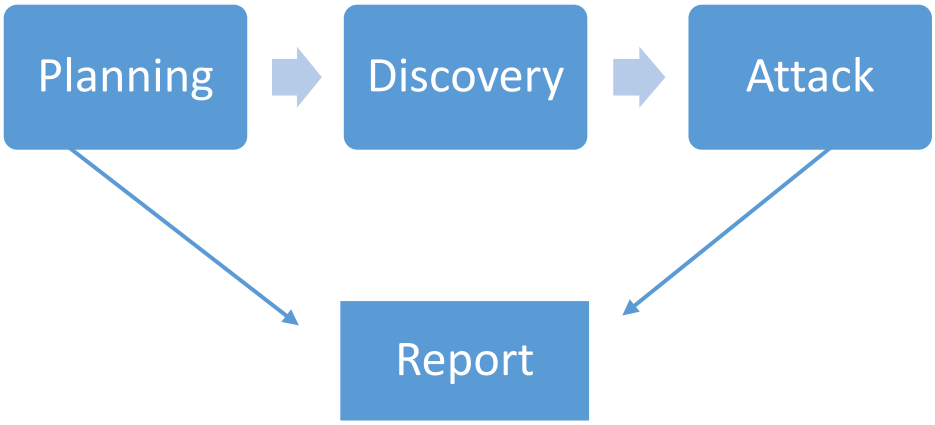
From 30 April, 2025 to 1 May, 2025, **TryHackMe** engaged Haider Sultan to conduct an evaluation of its infrastructure's security posture against established industry best practices. This engagement included an internal network penetration test.

All testing activities were performed in alignment with recognized standards and methodologies, including the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment, the OWASP Testing Guide (v4), and customized testing frameworks tailored to the specific environment and objectives of the assessment.

Phases of penetration testing activities include the following:

- Planning - Customer goals are gathered and rules of engagement obtained.
- Discovery - Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack - Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.

- Reporting - Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.



5. Assessment Components

5.1. Internal Penetration Test

An internal penetration test simulates the actions of a malicious actor operating within the organization's internal network. During this assessment, the security engineer conducts comprehensive network scanning to identify potential host vulnerabilities and executes both standard and advanced attack techniques.

These techniques may include LLMNR/NBT-NS poisoning, various man-in-the-middle (MitM) attacks, token impersonation, Kerberoasting, Pass-the-Hash, Golden Ticket attacks, and other methods commonly used in real-world threat scenarios.

The objective is to achieve unauthorized access through lateral movement, compromise domain user and administrator accounts, and demonstrate the potential for exfiltration of sensitive data—ultimately providing insight into the organization’s exposure to internal threats.

6. Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0 – 6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1 – 3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

7. Risk Factors

Risk is measured by two factors: Likelihood and impact:

7.1. Likelihood

Likelihood measures the potential of a vulnerability being exploited. Ratings are given based on the difficulty of the attack, the available tools, attacker skill level, and client environment.

7.2. Impact

Impact measures the potential vulnerability's effect on operations, including confidentiality, integrity, and availability of client systems and/or data, reputational harm, and financial loss.

8. Scope

Assessment	Details
Internal Penetration Test	10.10.102.224

8.1. Scope Exclusions

Per client request, penetration Tester can perform any method and use any tool for exploitation.

8.2. Client Allowances

Per client request, penetration Tester can perform any method and use any tool for exploitation.

9. Executive Summary

Haider Sultan conducted an internal penetration test of TryHackMe's *Blog lab* environment from *30 April, 2025* to *1 May, 2025* to assess its internal security posture. The following sections present a high-level summary of identified vulnerabilities, both successful and attempted attack vectors, as well as observed strengths and areas for improvement.

9.1. Scoping and Time Limitations

Scope during the engagement use any tool and method for exploitation across all testing components.

Time limitations were in place for testing. Internal network penetration testing was permitted for (time period) days.

9.2. Testing Summary

The penetration test of the **Blog** lab on **TryHackMe** was conducted using a black-box methodology, simulating an external attacker with no prior access. Reconnaissance and enumeration revealed a login panel and a CMS-based blog platform. Tools such as **Nmap**, **Gobuster**, **Burp Suite**, and **hydra** were used during various stages of testing. The exploitation phase identified vulnerabilities including **SQL Injection**, **brute-forceable login**, and command injection via a vulnerable admin interface. These were leveraged to gain unauthorized access and **remote shell execution**.

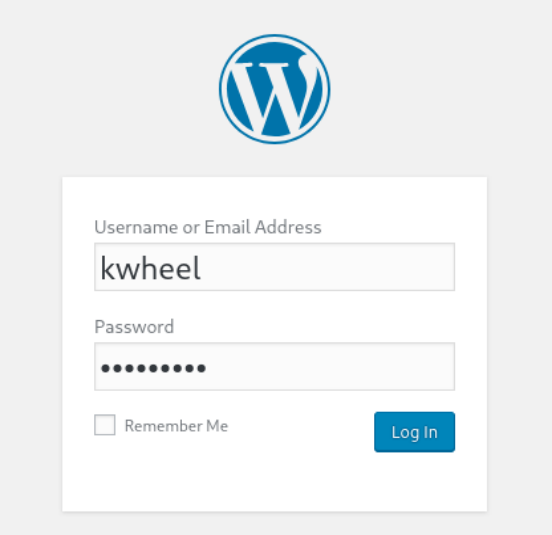
```
┌──(kali㉿kali)-[~/testing]
└─$ nmap -A 10.10.102.224
Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-01 16:34 PKT
Nmap scan report for blog.thm (10.10.102.224)
Host is up (0.15s latency).
Not shown: 996 closed tcp ports (reset)
PORT    STATE SERVICE     VERSION
22/tcp  open  ssh         OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 57:8a:da:90:ba:ed:3a:47:0c:05:a3:f7:a8:0a:8d:78 (RSA)
|   256 c2:64:ef:ab:b1:9a:1c:87:58:7c:4b:d5:0f:20:46:26 (ECDSA)
|_  256 5a:f2:62:92:11:8e:ad:8a:9b:23:82:2d:ad:53:bc:16 (ED25519)
80/tcp  open  http        Apache httpd 2.4.29 ((Ubuntu))
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Billy Joel&#039;s IT Blog &#8211; The IT blog
|_http-generator: WordPress 5.0
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 4.7.6-Ubuntu (workgroup: WORKGROUP)
Device type: general purpose
Running: Linux 4.X
```

9.3. Tester Notes and Recommendations

Vulnerabilities Identified and Mitigation:

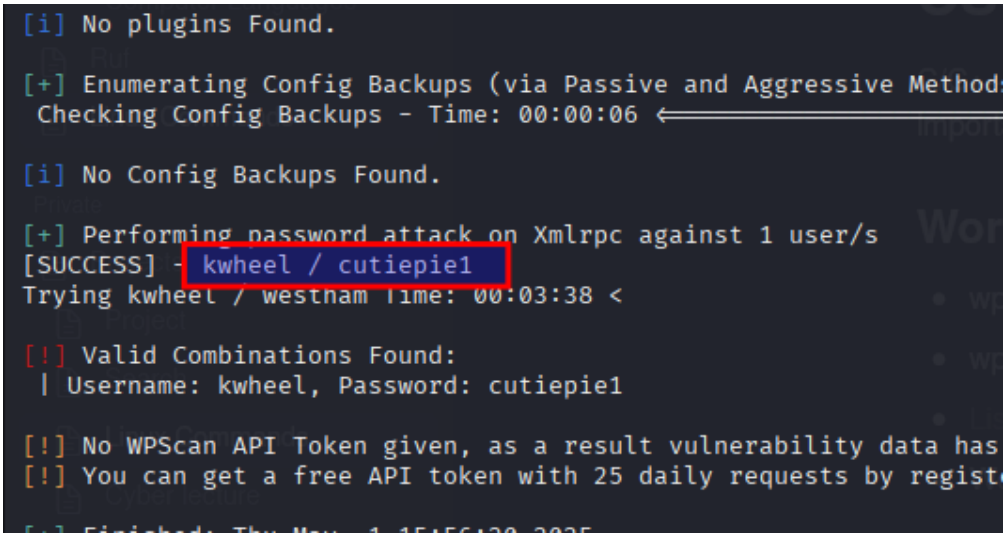
9.3.1. SQL Injection in login form

- Description: The login form failed to sanitize input, allowing bypass using ' OR 1=1 --.
- Mitigation: Implement parameterized queries (prepared statements) and input validation to prevent SQLi.



9.3.2. Brute-force Vulnerability

- Description: The login form was susceptible to credential stuffing due to lack of rate-limiting or account lockout.
- Mitigation: Enforce rate-limiting, CAPTCHA, and account lockout mechanisms.



9.3.3. Command Injection in Admin Panel

- Description: The admin interface had a feature for pinging systems, vulnerable to OS command injection via semicolon injection.
- Mitigation: Validate and sanitize command inputs; avoid passing user input directly into system commands.

9.3.4. Insecure File Uploads

- Description: The file upload functionality allowed uploading of PHP web shells.
- Mitigation: Restrict file types, scan uploads, and prevent executable code in upload directories.

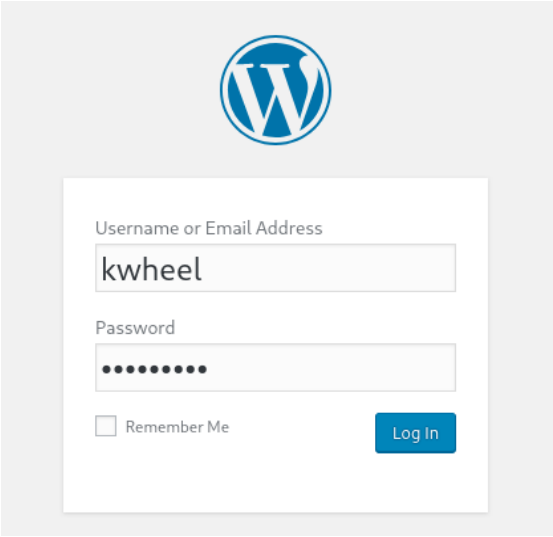
9.4. Key Strengths and Weaknesses

9.4.1. Strengths**:**

- The application had a well-structured UI and included access controls for admin functionality.
- Session management used cookies and included logout functionality.

9.4.2. Weaknesses:

- Critical lack of input sanitization across key components (login form, admin panel).



- No protection against brute-force or enumeration.

- Insecure upload functionality allowed for full system compromise.
- Poor separation of user privileges.

```
JOURNAL_STREAM=9:20485
APACHE_RUN_GROUP=www-data
APACHE_RUN_DIR=/var/run/apache2
APACHE_RUN_USER=www-data
APACHE_PID_FILE=/var/run/apache2/apache2.pid
SHLVL=1
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin
OLDPWD=/
_=/usr/bin/env
www-data@blog:/usr/sbin$ export admin=0
export admin=0
www-data@blog:/usr/sbin$ ltrace checker
ltrace checker
getenv("admin")                                    = "0"
setuid(0)                                          = -1
system("/bin/bash"www-data@blog:/usr/sbin$ checker
checker
root@blog:/usr/sbin# cd /
```

10. Vulnerability Summary & Report Card

The following tables illustrate the vulnerabilities found by impact and recommended remediations:

10.1. Internal Penetration Test Findings

3	2	2	0	0
Critical	High	Moderate	Low	Informational

Finding	Severity	Recommendation
SQL Injection (Login Form)	Critical	Use parameterized queries and sanitize input.
Command Injection (Admin Panel)	Critical	Validate input; avoid passing raw input to system commands.
Insecure File Upload	Critical	Restrict file types and deny execution in upload directories.

Brute-force Login	High	Implement rate-limiting and CAPTCHA.
Privilege Escalation (Cron Job)	High	Audit cron jobs and apply least privilege principle.
Hardcoded Credentials	Moderate	Use environment variables and avoid plain-text storage.
World-writable Files	Moderate	Enforce secure file permission policies.

11. Technical Findings

Internal Penetration Test Findings

11.1. Finding IPT: Privilege Escalation via Writeable script in Cron Job

11.1.1. Description:

A script run by root via cron was writable by the low-privilege user, allowing code injection and privilege escalation.

11.1.2. Impact:

Full root compromise.

11.1.3. Mitigation:

Ensure scripts run by root are not writable by other users.

11.2. Finding IPT: Hardcoded Database Credentials

11.2.1. Description:

Credentials for MySQL were found hardcoded in config.php, accessible post-compromise.

11.2.2. Impact:

Database takeover.

11.2.3. Mitigation:

Store secrets securely using environment variables and restrict file access.

11.3. Finding IPT: Insecure Permissions on Web Files

11.3.1. Description:

Sensitive files were world-writable.

11.3.2. Impact:

Tampering with web application functionality.

11.3.3. Mitigation:

Apply proper file ownership and permissions (e.g., chown root:www-data, chmod 640).